

Module 9 — Full-Stack App (Bringing It All Together)

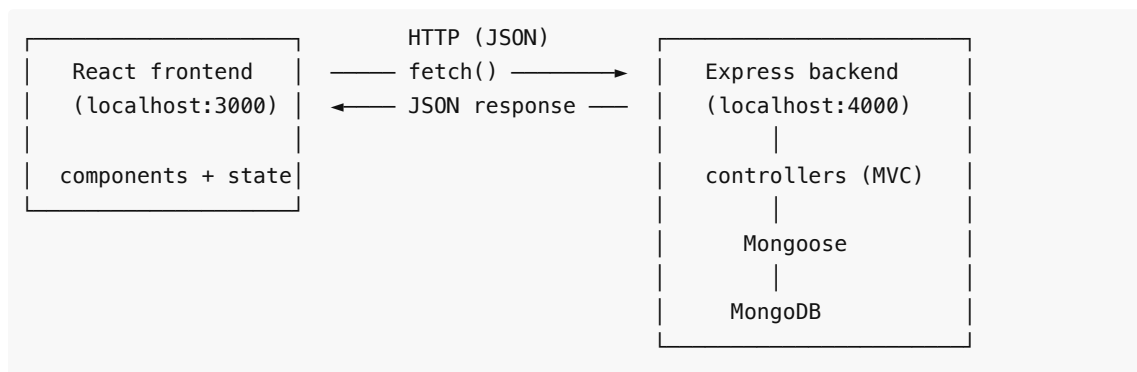
Time: 1.5 hours · Covers Practical 10 · CO5

Practical 10: Development of a full stack web application with a combination of all.

This is the capstone. We combine **everything**: a React frontend (Modules 6–8) talking over **HTTP** (Module 2) to a **Node/Express + MongoDB** backend built with the **MVC + CRUD** pattern (Modules 3–4).

We build a **Task Manager**: view tasks, add tasks, toggle done, delete.

Part A — Full-stack architecture (concept, ~15 min)



Two separate programs:

- **Backend** — an API server (the CRUD+MVC app from Module 4, trimmed to tasks). Owns the database. Runs on port **4000**.
- **Frontend** — a React app. Owns the UI. Runs on port **3000**. It never touches the database directly; it only calls the backend's HTTP API.

This separation is how most real web apps are built.

Part B — The data flow (concept, ~15 min)

Trace **adding a task** end to end:

1. User types a title and clicks **Add** in a React form (Module 8).
2. Frontend calls `POST http://localhost:4000/api/tasks` with JSON (`fetch` , Module 2).
3. Express **route** → **controller** (Module 4) → **Mongoose model** saves it to **MongoDB** (Modules 3).
4. Backend responds `201 Created` with the new task as JSON.
5. Frontend puts the returned task into **state** → the list re-renders (Module 7). The new task appears — **seamless data flow (CO5)**.

Part C — CORS: letting the two talk (~10 min)

The frontend (`:3000`) and backend (`:4000`) are different origins. Browsers block cross-origin requests by default. The backend enables **CORS** to allow the frontend to call it:

```
const cors = require("cors");
```

```
app.use(cors()); // allow the React dev server to call this API
```

Part D — Run both halves (~30 min)

Open **two terminals**.

Terminal 1 — backend:

```
cd 09-fullstack/backend
npm install
cp .env.example .env      # set your MongoDB connection string
npm start                 # API on http://localhost:4000
```

Terminal 2 — frontend:

```
cd 09-fullstack/frontend
npm install
npm start                # UI on http://localhost:3000
```

Open <http://localhost:3000>. Add, toggle, and delete tasks. Refresh the page — your tasks persist because they live in MongoDB, not in browser memory.

Part E — Reading the code (~20 min)

Backend (same MVC shape as Module 4):

- [backend/src/models/task.model.js](#) — Model
- [backend/src/controllers/task.controller.js](#) — Controller
- [backend/src/routes/task.routes.js](#) — Routes
- [backend/src/app.js](#) — app + CORS
- [backend/server.js](#) — startup

Frontend:

- [frontend/src/api/tasksApi.js](#) — all `fetch` calls in one place (the frontend's "data layer")
- [frontend/src/components/TaskForm.js](#) — add a task (props + state, Module 8)
- [frontend/src/components/TaskList.js](#) — render tasks (list + props, Module 7)
- [frontend/src/App.js](#) — loads tasks with `useEffect`, owns the state, wires it together

New hook: `useEffect`

To fetch data when the page first loads, we use `useEffect` :

```
useEffect(() => {
  loadTasks(); // runs once after the first render
}, []); // empty [] = "run only once"
```

Summary

- A full-stack app = **frontend (React)** + **backend (Express/Mongo API)**, talking over **HTTP/JSON**.

- Frontend owns the UI + state; backend owns the data + business logic (MVC).
- **CORS** lets the two origins communicate in development.
- **useEffect** loads data on mount; state updates keep the UI in sync — seamless data flow (CO5).
- Every earlier module shows up here: Node, HTTP/status codes, Mongo models, MVC/CRUD, React components, props, state, forms.

Now do [practice.md](#) .